

An Incentive Framework for Task Offloading in Edge Computing Marketplaces Under Price Competition

Liantao Wu[✉], Peng Sun[✉], Zhibo Wang[✉], *Senior Member, IEEE*, Xiaoyi Pang[✉], Jiahui Hu[✉],
Honglong Chen[✉], *Senior Member, IEEE*, and Yang Yang[✉], *Fellow, IEEE*

I. INTRODUCTION

Abstract—To efficiently execute tasks, computation resource requesters (CRRs) with limited resources can offload their tasks to nearby computation resource providers (CRPs) with spare computing capacity. These CRPs require appropriate incentives to compensate for their incurred costs when helping process the offloaded tasks. Although several mechanisms have been designed to incentivize CRPs, none of them have investigated the incentive mechanism considering price-setting and price-taking CRPs simultaneously. In this work, we propose an incentive framework for task offloading in the edge computing marketplace that includes both price-setting and price-taking CRPs. We model the CRR's interactions with both types of CRPs as a three-stage Stackelberg game to maximize the profit for both the CRR and CRPs. We prove the existence of a unique subgame perfect equilibrium (SPE) of the formulated game and further develop iterative algorithms for the CRR and price-setting CRPs to achieve the equilibrium. Through the designed algorithms, each CRP does not require complete information about the CRR and other CRPs. Extensive simulations demonstrate that offloading tasks to both price-setting and price-taking CRPs achieves higher profits for the CRR and price-setting CRPs compared to offloading tasks solely to price-setting CRPs. Additionally, the obtained SPE can achieve near-optimal social welfare.

Index Terms—Edge computing, incentive mechanism, task offloading, Stackelberg game, subgame perfect equilibrium.

Received 13 March 2024; revised 15 February 2025; accepted 26 March 2025. Date of publication 31 March 2025; date of current version 6 August 2025. This work was supported in part by NSFC under Grant 62202307, Grant 62472158, Grant U24B20182, Grant 62102337, and Grant 62111530052, in part by the Natural Science Foundation of Hunan Province under Grant 2023JJ40174, in part by the Science and Technology Innovation Program of Hunan Province under Grant 2024RC3102, in part by Young Elite Scientists Sponsorship Program by CAST under Grant 2023QNRC001, in part by Shandong Provincial Taishan Scholar Program under Grant tsqn202312133, in part by Shandong Provincial Natural Science Foundation under Grant ZR2022YQ61, in part by the Open Topics from the Lion Rock Labs of Cyberspace Security under Grant LRL24015, and in part by the Fundamental Research Funds for the Central Universities Shanghai Natural Science Foundation under Grant 23ZR1417800. Recommended for acceptance by L. Duan. (*Corresponding author: Peng Sun.*)

Liantao Wu is with the Shanghai Key Laboratory of Trustworthy Computing, Software Engineering Institute, East China Normal University, Shanghai 200062, China (e-mail: ltwu@sei.ecnu.edu.cn).

Peng Sun is with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China (e-mail: psun@hnu.edu.cn).

Zhibo Wang, Xiaoyi Pang, and Jiahui Hu are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China (e-mail: zhibowang@zju.edu.cn; xypang@zju.edu.cn; jiahuihu@zju.edu.cn).

Honglong Chen is with the College of Control Science and Engineering, China University of Petroleum (East China), Qingdao 266580, China (e-mail: chenhl@upc.edu.cn).

Yang Yang is with the IoT Thrust, Hong Kong University of Science and Technology, Guangzhou 510000, China (e-mail: yyiot@hkust-gz.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TMC.2025.3556516>, provided by the authors.

Digital Object Identifier 10.1109/TMC.2025.3556516

WITH the proliferation of smart devices (e.g., smartphones and smart glasses), a variety of services and applications, such as facial recognition, virtual reality, and interactive gaming, have emerged and significantly enriched people's lives [1], [2]. However, these applications are typically computation-intensive and delay-sensitive. Unfortunately, smart devices usually have limited resources and are unable to fully fulfill the requirements of these applications [3], [4]. Multi-access edge computing (MEC) is recognized as a promising solution for providing a high quality of service (QoS) for these demanding applications [5], [6], [7]. Several telecom operators and technology industry leaders, such as China Mobile and Microsoft, have already launched commercial edge computing services [8]. Besides offloading tasks to edge computing servers, users can also offload tasks to nearby devices with idle computing resources through device-to-device (D2D) offloading [9], [10], [11]. In this scenario, both edge servers and idle mobile users can serve as computation resource providers (CRPs), while users offloading their tasks to CRPs for accelerated processing are categorized as computation resource requesters (CRRs). For non-splittable tasks, the task can either be executed locally or by a CRP, while for splittable tasks, i.e., tasks can be divided into multiple parts, they can be offloaded to multiple CRPs to further reduce the execution delay using parallel processing.

For CRPs, offering offloading services to CRRs incurs costs such as operating expenses and energy consumption [8]. An appropriate incentive mechanism needs to be designed to compensate participating CRPs and offset their incurred costs. Existing incentive mechanisms for task offloading in edge computing [12], [13], [14], [15], [16] typically assume that CRPs, for instance, servers deployed by major telecom operators and technology giants, possess greater market power than CRRs. Hence, CRPs set their prices, and CRRs can only accept these prices to offload their tasks to selected CRPs. However, in practice, some CRPs, such as idle mobile users, may have low market power, and therefore, they are price-takers instead of price-setters. CRRs can set prices when offloading their tasks to these CRPs with low market power rather than just accepting the offered prices. For example, in decentralized computing resource markets, e.g., Golem Network¹, the CRR can set the price they are willing to pay for the computing resources and, accordingly, the CRPs become price-takers. In this case, the aforementioned

¹ <https://www.golem.network/>

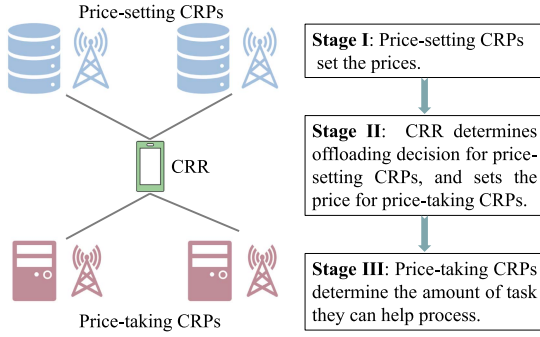


Fig. 1. The three-stage Stackelberg game model for task offloading in an edge computing marketplace.

existing models are no longer applicable for operating such edge computing markets. Therefore, it is crucial to determine the optimal approach for the CRR to interact with both price-setting and price-taking CRPs in an edge computing marketplace.

Building upon the discussions above, in this work, we propose an incentive framework for splittable task offloading in an edge computing marketplace that allows the CRR to interact with both price-setting and price-taking CRPs. We classify individual idle mobile users or servers owned by small business entities as price-taking CRPs. The CRR has the priority to set prices for these CRPs. Conversely, price-setting CRPs possess more market power than the CRR. For example, edge servers deployed by major telecom operators and tech giants possess greater market power than a single mobile user making a task-offloading request. Therefore, our work distinguishes itself from existing literature in the following two fundamental aspects:

- The CRR engages in negotiations with both price-setting and price-taking CRPs.
- The CRR's main objective is to maximize its profit by reducing the task execution time while minimizing the payments to CRPs and the cost of local computing.

To facilitate the analysis, we model the CRR's interactions with price-setting and price-taking CRPs as a three-stage Stackelberg game, as illustrated in Fig. 1, and focus on maximizing profits for both the CRR and the CRPs. Through the formulated game, we investigate the price competition among price-setting CRPs and the competition between price-taking CRPs regarding the percentage of the task they can process for the CRR. In the first stage, price-setting CRPs strategically set their prices to maximize their individual payoffs. In the second stage, the CRR makes a decision regarding whether to engage the price-setting CRPs by either accepting or declining their proposed prices. In case of acceptance, the CRR also determines the amount of offloaded tasks to them. Furthermore, in this stage, the CRR sets prices for the price-taking CRPs, aiming to maximize its own profit. Finally, price-taking CRPs follow the CRR's prices to determine the percentage of the task they will process for the CRR to maximize their own payoffs in the third stage.

In summary, this paper makes the following contributions:

- *A Novel Incentive Framework for Edge Computing Marketplaces:* To the best of our knowledge, we propose the first incentive framework for task offloading in an edge

computing marketplace that incorporates both price-setting and price-taking CRPs. The interactions between the CRR and the two types of CRPs are systematically modeled as a three-stage Stackelberg game. Besides, we derive the corresponding subgame perfect equilibrium, which provides insights into market prices.

- *Rigorous Equilibrium Analysis:* We conduct a thorough analysis of the formulated game, demonstrating its convex nature and proving the existence of a subgame perfect equilibrium. Furthermore, we demonstrate that the CRR's profit maximization problem yields a unique equilibrium strategy.
- *Efficient Algorithm Design:* We propose iterative algorithms to achieve the subgame perfect equilibrium. One algorithm is used to determine the optimal response strategies for price-setting CRPs, and another is designed to facilitate information exchange between the CRR and the CRPs. Based on these proposed algorithms, the price-setting CRPs can avoid the necessity of obtaining full information about the CRR and other CRPs while incurring only a small communication overhead. We provide proof of convergence for the proposed algorithms to the unique subgame perfect equilibrium strategy.
- *Extensive Performance Evaluation:* We conduct extensive numerical experiments to demonstrate that offloading to both price-taking and price-setting CRPs can yield higher profits for the CRR and price-setting CRPs compared to offloading only to price-setting CRPs. Furthermore, we show that the social welfare of the subgame perfect equilibrium derived in our proposed framework, where the CRR offloads its task to both price-setting and price-taking CRPs, is close to the socially optimal solution.

Paper Organization: We first review and discuss related works in Section II. Section III presents the system model, where the three-stage Stackelberg game is formulated. In Section IV, we employ backward induction method to derive the corresponding solutions to the game. We design iterative algorithms for price-setting non-cooperative game (PS-NCG) in Section V. We provide numerical results in Section VI. Finally, Section VII concludes the paper.

II. RELATED WORK

In this section, we review and discuss related works from two aspects, i.e., task offloading in edge computing and incentive mechanism design for edge computing, especially incentive mechanism design for task offloading.

A. Task Offloading in Edge Computing

MEC, as a novel computing paradigm, offers computation, communication, and storage resources to CRRs. By offloading tasks to CRPs, MEC effectively mitigates the computational burden on CRRs, thereby enhancing their QoS.

Early studies focused on scenarios involving a single CRP (e.g., an MEC server). Chen et al. in [17] investigated the multi-user offloading problem within a multi-channel wireless interference environment. They formulated it as a multi-user

offloading game and devised a distributed algorithm to achieve a Nash equilibrium. Kuang et al. in [18] studied the problem of partial offloading scheduling and power allocation to minimize the weighted sum of energy cost and task latency. They introduced an iterative optimization algorithm to address this problem. Wang et al. in [19] developed a non-cooperative offloading game for multiple users in vehicular MEC networks. They introduced a distributed best-response algorithm under the offloading game model to maximize each vehicle's utility.

Subsequently, the paradigm of task offloading from mobile users to multiple CRPs, including MEC servers and idle mobile users, has gained substantial attention. Researchers in [20], [21] explored scenarios where a single CRR offloads tasks to multiple CRPs. Zhang et al. in [20] introduced an offloading algorithm that emphasizes fairness and energy minimization. This algorithm takes into account factors such as energy cost, historical average energy, and mobile user priority. Ko et al. in [21] proposed a distributed device-to-device offloading scheme, wherein a CRR opportunistically broadcasts offloading requests. CRPs employ a constrained stochastic game to collectively decide whether to accept these requests in a distributed manner. Zhang et al. in [22] investigated task offloading from multiple users to multiple servers in an energy-harvesting-aided MEC environment based on game theory. The objective is to minimize the users' average task latency, including waiting time, communication time, and processing time.

The aforementioned studies primarily focus on binary or partial offloading, often overlooking the concept of splittable task offloading. It is worth noting that applications centered around data partitioning can be divided into multiple subtasks to facilitate parallel processing. Liu et al. in [23] investigated a parallel offloading problem of splittable tasks, formulating it as a generalized Nash equilibrium problem. They proved the existence of a generalized Nash equilibrium and developed a distributed offloading algorithm. Peng et al. in [24] studied the partial offloading problem of multiple splittable tasks to minimize total user latency. The optimal solution to this problem is obtained by constructing a linear system of equations based on the barrel theory.

In this work, we focus on splittable task offloading, where a single CRR offloads its tasks to multiple CRPs, including both price-taking and price-setting CRPs, to maximize its profit.

B. Incentive Mechanisms for Edge Computing

CRPs incur additional costs, such as operating expenses and energy consumption costs, when assisting CRRs in processing tasks. Therefore, it becomes imperative to establish an incentive mechanism that adequately compensates CRPs and offsets their associated costs.

Previous research has explored incentive mechanisms in scenarios where multiple users offload tasks to a single MEC server [12], [13], [14]. For instance, Liu et al. in [12] presented a price-based offloading strategy that takes into account computation capacity constraints. They employed a Stackelberg game to model the interactions between the edge server and users.

Tao et al. in [13] studied a Stackelberg game-based pricing mechanism for partial offloading, with a focus on the interplay between the server's resource pricing and task offloading quantities. Tong et al. in [14] formulated the resource pricing and task offloading problem under edge servers' resource constraints as a Stackelberg game. They proposed a differentiated pricing algorithm based on particle swarm optimization to optimize the server's pricing.

Recently, researchers have designed several incentive mechanisms for task offloading in multi-server edge computing scenarios [8], [10], [11], [15], [16]. For example, Pu et al. in [10] considered collaborative task offloading, where idle mobile users and edge servers contribute their spare computing resources to process mobile users' tasks. Li et al. in [11] tackled a social welfare maximization problem that involves collaborator selection, resource allocation, time scheduling, and pricing policy design. This problem was addressed using a primal-dual optimization framework.

Furthermore, recent research has explored the competition among multiple edge servers deployed by different business entities [25], [26]. For instance, Chen et al. in [8] investigated the price competition of heterogeneous edge servers by modeling the interactions between edge servers and users as a two-stage game, where edge servers with diverse costs competitively set their prices, and users make self-interested selections of edge servers to offload their tasks. Wang et al. in [15] designed an online profit maximization multi-round auction mechanism for resource trading between edge servers and mobile users in a competitive environment. This mechanism determines the price paid by users to utilize the resources provided by servers. Wang et al. in [16] studied hierarchical dynamic pricing for cloud-edge-client collaboration, utilizing a double-layer Stackelberg game model to describe the mixed game process of cloud servers, edge servers, and mobile users.

However, these incentive mechanisms for task offloading often assume that CRPs hold greater market power than CRRs. In such scenarios, CRPs set the prices, and CRRs can only accept these prices to offload their tasks to selected CRPs. Nevertheless, in real-world settings, certain CRPs, such as idle mobile users, may possess limited market power and operate as price-takers rather than price-setters. Consequently, CRRs can set prices when offloading their tasks to these CRPs with lower market influence. Although Wang et al. in [27], [28] investigated the scenario where different resource providers (RP) may hold different market power, they did not explicitly examine how the relative market power between RPs and the resource requester (RR) affects interactions between RPs with different levels of market power and the RR, nor did they address the implications for pricing priorities. In this context, the existing models mentioned above are no longer suitable for managing such markets. A few prior studies have explored the existence of two types of RPs in data offloading market [29] and data marketplace with differentially private federated learning [30]. However, the problems addressed and the system models formulated in these works are not well-suited for edge computing marketplaces. To address above challenges, our work introduces a novel task offloading marketplace that formally models the

TABLE I
KEY NOTATIONS

Symbol	Definition
N	number of CRPs
N^s	number of price-setting CRPs
N^t	number of price-taking CRPs
$\mathbf{x}^s$	offloaded task percentage vector of price-setting CRPs
$\mathbf{x}^t$	offloaded task percentage vector of price-taking CRPs
x_n^s	percentage of CRR's task offloaded to price-setting CRP n
x_n^t	percentage of CRR's task offloaded to price-taking CRP n
$\mathbf{p}^s$	price vector of price-setting CRPs
$\mathbf{p}^t$	price vector of price-taking CRPs
p_n^s	price set by price-setting CRP n
p_n^t	price set by CRR for price-taking CRP n
x_0	percentage of CRR's task executed locally
d	computation workload required to execute CRR's task
c_n^s	task cost of price-setting CRP n when processing CRR's task
c_n^t	task cost of price-taking CRP n when processing CRR's task
α_n^s	cost factor of price-setting CRP n
α_n^t	cost factor of price-taking CRP n
α_0	cost factor of CRR
q_n^s	CRR's payment function for price-setting CRP n
q_n^t	CRR's payment function for price-taking CRP n
V_n^s	profit of price-setting CRP n
V_n^t	profit of price-taking CRP n
V	profit of CRR
f_n^s	computation resources allocated by price-setting CRP n
f_n^t	computation resources allocated by price-taking CRP n
c	CRR's utility constant
$\bar{\tau}$	average computation delay
T_s	saved execution time by offloading splittable tasks to CRPs
f_0	computing power of CRR
$\mathbf{p}_{-n}^s$	strategy of the price-setting CRPs excluding n
$\eta^{(t)}$	step size at iteration t in Algorithm 1
ϵ	termination threshold in Algorithm 1
ϵ'	termination threshold in Algorithm 2

interactions between CRRs and both price-setting and price-taking CRPs.

III. SYSTEM MODEL AND GAME FORMULATION

In this section, we first present a high-level overview of the studied edge computing marketplace in Section III-A, including its components and workflow. Then, we respectively characterize the profit of CRR and CRPs in Section III-B. Finally, in Section III-C, we formulate the three-stage game.

A. System Overview

For clarity, we first summarize the key notations used in this paper in Table I.

As shown in Fig. 2, we consider an edge computing marketplace comprising a CRR and a set $\mathcal{N} = \{1, 2, \dots, N\}$ of heterogeneous CRPs. These CRPs possess varying processing capabilities and provide task offloading services to the CRR. There are two types of CRPs in the market, i.e., a set $\mathcal{N}^s = \{1, 2, \dots, N^s\}$ of price-setting CRPs and a set $\mathcal{N}^t = \{1, 2, \dots, N^t\}$ of price-taking ones. Thus, the set $\mathcal{N}$ is the union of $\mathcal{N}^t$ and $\mathcal{N}^s$ (note that $\mathcal{N}^t \cap \mathcal{N}^s = \emptyset$).

The task of the CRR is both time-sensitive and computationally intensive, and thus, it would consider offloading the task to the two types of CRPs. In this work, we focus on data partitioned-oriented tasks that can be divided into separate segments for parallel processing. Examples of such tasks include

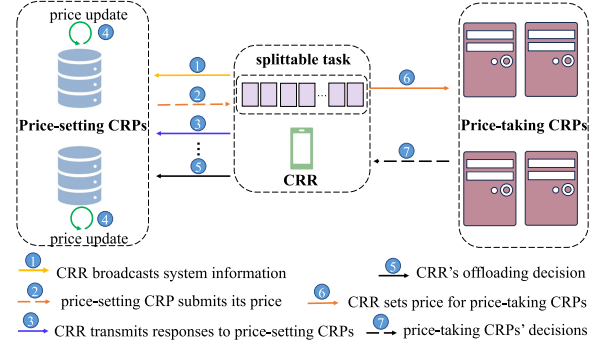


Fig. 2. An edge computing marketplace with one CRR, two price-setting CRPs, and four price-taking CRPs.

visual applications, photosynth, Gzip compression, and virus scanning [10].

To encourage CRPs to assist in task processing, the CRR must offer suitable economic incentives to them to compensate for their incurred costs. In the case of CRPs belonging to major telecom operators (e.g., Verizon and China Mobile), the CRPs set the prices, making them price-setting CRPs. Conversely, for CRPs with limited market influence, such as idle mobile devices, they act as price-takers.

As shown in Fig. 2, the workflow of the considered edge computing marketplace is described as follows:

- 1) The CRR collects and broadcasts necessary system information for interactions with price-setting CRPs (step &z.oone;). The necessary system information includes the set of price-setting CRPs, the CRR's utility constant, and the computing power and cost factors of the CRR and CRPs.
- 2) Price-setting CRPs submit their pricing proposals to the CRR. Initially, these proposals are randomly selected, and subsequent prices are updated according to the response from the CRR (step &z.otwo;).
- 3) Using the prices submitted by price-setting CRPs, the CRR determines the corresponding task offloading percentage to price-setting CRPs and transmits this information to them as well as the derivative of the offloaded task percentage with respect to price (step &z.othr;).
- 4) Price-setting CRPs update their prices based on the CRR's responses using a gradient-based method (step &z.ofour;).
- 5) Step &z.otwo; to step &z.ofour; are repeated until the difference between two consecutive updated prices falls below a predetermined threshold. After convergence, the CRR determines the task offloading percentage to price-setting CRPs (step &z.ofive;).
- 6) The CRR sets the price for price-taking CRPs based on the system information and the prices proposed by price-setting CRPs. (step &z.osix;)
- 7) Price-taking CRPs determine the portion of the CRR's task they will assist in processing (step &z.osev;).

The strategic interaction between price-setting CRPs and the CRR is modeled as a price-setting non-cooperative game, where one price-setting CRP competes with the others to attract the CRR's task offloading. Price-setting CRPs adjust their prices

iteratively based on the CRR's responses. The iterative updates enable price-setting CRPs and the CRR to converge to a Nash Equilibrium.

On the other hand, price-taking CRPs lack the market power to influence the price. They simply accept the price set by the CRR and decide their response. Since there is no competition among price-taking CRPs, and they cannot adjust prices, the interaction is non-strategic and resolved in a single step. Thus, no iterative updates are necessary between the CRR and price-taking CRPs.

B. Modeling of CRPs and the CRR

We first model the profit of price-taking and price-setting CRPs and the CRR. Subsequently, we proceed to model the CRR's interactions between CRPs as a three-stage Stackelberg game, where each player determines their strategy to maximize their own profit.

In this work, we focus on splittable task offloading. We leverage the variables x_n^t and x_n^s to respectively represent the percentage of the CRR's task offloaded to and processed on price-taking CRP n and price-setting CRP n . Additionally, x_0 stands for the percentage of the CRR's task executed locally. Obviously, we have

$$\sum_{n \in \mathcal{N}^t} x_n^t + \sum_{n \in \mathcal{N}^s} x_n^s + x_0 = 1. \quad (1)$$

1) CRPs' Profit:

- 1) *Cost*: CRPs need to allocate a specific amount of computation resources when assisting in processing the CRR's task. Accordingly, CRPs will incur marginal costs (in dollars per CPU cycle), e.g., operating expenses or power consumption. According to a previous study [14], the marginal cost of the CRPs can be approximated using strictly convex functions, such as the quadratic function. Specifically, if we let d represent the computation workload (in CPU cycles) required to execute the CRR's task, the incurred costs of price-taking and price-setting CRPs are respectively represented as

$$c_n^t(x_n^t) = \alpha_n^t (x_n^t d)^2 \quad (2)$$

and

$$c_n^s(x_n^s) = \alpha_n^s (x_n^s d)^2, \quad (3)$$

where α_n^t and α_n^s represent the cost factors for price-taking and price-setting CRPs, respectively.

- 2) *Payment*: To compensate for CRPs' incurred costs, they need to receive proper payments from the CRR. Price-setting CRPs have the priority to set such a payment, whereas price-taking CRPs can only decide whether to accept or reject the payment determined by the CRR. Price-setting CRPs' cost structures are often nonlinear, with increasing marginal costs as demand grows due to factors like network congestion, resource inefficiencies, or infrastructure investments. To capture this complexity, we use a quadratic payment function, which aligns with real-world scenarios [31]. In contrast, price-taking CRPs are smaller entities with limited market power, accepting

prices set by the CRR. Their costs are more proportional to the workload, typically increasing linearly with demand. Thus, a linear payment function appropriately reflects their cost structure and market behavior. Therefore, this paper employs the following distinct payment functions to reflect the varying market power of different CRPs.

- *Price-setting CRPs*: The CRR's payment function for CRP $n \in \mathcal{N}^s$ is $q_n^s(x_n^s) = p_n^s (x_n^s d)^2$ with p_n^s being the price set by the CRP n .
- *Price-taking CRPs*: The CRR's payment function for CRP $n \in \mathcal{N}^t$ is $q_n^t(x_n^t) = p_n^t (x_n^t d)$, where p_n^t represents the price set by the CRR.

Therefore, the price-taking and price-setting CRPs' profits (i.e., received payments minus incurred costs) are as follows:

$$V_n^t(x_n^t, p_n^t) = p_n^t (x_n^t d) - \alpha_n^t (x_n^t d)^2, n \in \mathcal{N}^t, \quad (4)$$

$$V_n^s(x_n^s, p_n^s) = p_n^s (x_n^s d)^2 - \alpha_n^s (x_n^s d)^2, n \in \mathcal{N}^s, \quad (5)$$

where $p_n^s \geq \alpha_n^s$, which guarantees the payoff of price-setting CRPs is non-negative.

2) *The CRR's Profit*: The CRR's profit is usually defined as the difference between the utility obtained based on the satisfaction with the task offloading operation and the cost for computing resource usage [32].

- 1) *Utility*: To better measure the subjective satisfaction of the CRR with the task offloading operation, we first introduce the concept of the satisfaction of CRR (SoC) [33]. The SoC function quantitatively describes CRR's satisfaction with the task offloading activity according to the offloading decisions. Following previous works [13], the SoC function can be expressed as cT_s , where c is a utility constant, and T_s represents the saved execution time by offloading splittable tasks to CRPs. The task execution delay includes transmission delay and computation delay. Since CRPs, such as edge servers, are usually in close proximity to the CRR, the transmission delay is negligible [8]. Therefore, we focus on the computation delay. For local computing, the computation delay is calculated as d/f_0 , with f_0 being the computing power of the CRR. When the CRR offloads the splittable task to both price-setting and price-taking CRPs, the average computation delay is

$$\bar{\tau} = \frac{1}{N+1} \left(\sum_{n \in \mathcal{N}^t} \frac{x_n^t d}{f_n^t} + \sum_{n \in \mathcal{N}^s} \frac{x_n^s d}{f_n^s} + \frac{x_0 d}{f_0} \right),$$

where f_n^s and f_n^t denote the computation resources allocated to the CRR by price-setting and price-taking CRPs, respectively. Therefore, the utility function can be described as

$$u(\mathbf{x}) = c \left(\frac{d}{f_0} - \bar{\tau} \right), \quad (6)$$

where $\mathbf{x} \triangleq (x^t, x^s, x_0)$ with $x^s \triangleq (x_n^s : \forall n \in \mathcal{N}^s)$ and $x^t \triangleq (x_n^t : \forall n \in \mathcal{N}^t)$.

- 2) *Cost*: The CRR incurs costs when consuming computation resources for processing the task, which consists of two parts:

- *Cost in local computing:* With x_0 denoting the percentage of the CRR's task executed locally, the CRR's local computing cost can be expressed as $\alpha_0(x_0d)^2$, where α_0 represents the cost factor for the CRR.
- *Cost in task offloading:* When the CRR offloads its task to CRPs, the primary cost in completing the task is the payment reimbursed to the CRPs.

Therefore, by subtracting the local cost and payments to CRPs from the obtained utility, the CRR's profit is

$$V(\mathbf{x}, \mathbf{p}) = u(\mathbf{x}) - \alpha_0(x_0d)^2 - \sum_{n \in \mathcal{N}^s} p_n^s(x_n^s d)^2 - \sum_{n \in \mathcal{N}^t} p_n^t x_n^t d, \quad (7)$$

where $\mathbf{p} \triangleq (\mathbf{p}^t, \mathbf{p}^s)$, $\mathbf{p}^s \triangleq (p_n^s : \forall n \in \mathcal{N}^s)$ and $\mathbf{p}^t \triangleq (p_n^t : \forall n \in \mathcal{N}^t)$.

C. Splittable Task Offloading Game

We model the interactions between the CRR and the two types of CRPs as a three-stage Stackelberg game. In each of the stages, each player determines its strategy to maximize its own profit. We define the following game:

- *Stage I: Players:* price-setting CRPs $n \in \mathcal{N}^s$; *Strategy:* price vector $\mathbf{p}^s$; *Payoff:* $V_n^s(x_n^s, p_n^s)$ given in (5).
- *Stage II: Players:* CRR; *Strategy:* the vector of offloading percentage of the task for price-setting CRPs $\mathbf{x}^s$, and price vector $\mathbf{p}^t$ for price-taking CRPs; *Payoff:* $V(\mathbf{x}, \mathbf{p})$ given in (7).
- *Stage III: Players:* price-taking CRPs $n \in \mathcal{N}^t$; *Strategy:* the vector of offloading percentage of the CRR's task for price-taking CRPs $\mathbf{x}^t$; *Payoff:* $V_n^t(x_n^t, p_n^t)$ given in (4).

We aim to determine the subgame perfect equilibrium (SPE) of the multi-stage game, ensuring that neither the CRR nor the CRPs have incentives to unilaterally deviate from this equilibrium. In the next section, we will employ the backward induction method [34] to derive the corresponding solutions to the three-stage Stackelberg game.

IV. EQUILIBRIUM SOLVING AND ANALYSIS

In this section, we adopt the backward induction-based method to derive the solutions to the three-stage Stackelberg game. Subsequently, we study the existence and uniqueness of equilibrium in the PS-NCG of Stage I.

A. Backward Induction-Based Solutions

Before deriving the equilibrium, we first present the definition of subgame perfect equilibrium of the three-stage Stackelberg game.

Definition 1: (Subgame Perfect Equilibrium) A strategy profile $(\mathbf{x}^{NE}, \mathbf{p}^{NE})$ including the offloading strategies $\mathbf{x}^{NE}$ and price vector $\mathbf{p}^{NE}$ is a subgame perfect equilibrium if it represents a Nash equilibrium (NE) in every subgame of the original game.

In the following, we start with analyzing the price-taking CRPs' behavior in Stage III. We then obtain the strategy of the CRR in Stage II. Finally, we investigate the strategies of the price-setting CRPs in Stage I.

1) *Stage III (Price-Taking CRPs):* With the price-vector $\mathbf{p}^t$ given by the CRR, the price-taking CRPs determine the percentage of the task that they can help process in order to maximize their own profits in Stage III. In particular, each CRP $n \in \mathcal{N}^t$ selects its strategy x_n^t within the strategy space $[0, 1]$ to maximize its profit $V_n^t(x_n^t, p_n^t)$. This leads to the following optimization problem:

$$\max_{x_n^t} V_n^t(x_n^t, p_n^t) \quad (8)$$

$$\text{s.t. } 0 \leq x_n^t \leq 1, \quad (8a)$$

$$\frac{x_n^t d}{f_n^t} \leq \frac{d}{f_0}, \quad (8b)$$

where the constraint (8b) guarantees that the computation delay of price-taking CRP n is not higher than that of local computing. Since edge servers or idle mobile devices usually possess more powerful computing capabilities or more redundant computation resources, we can assume that f_n^t is larger than f_0 . Hence, (8b) is always satisfied, and we will omit it hereafter.

We can demonstrate that problem (8) is a concave maximization problem. In Theorem 1 below, we present its unique optimal solution, which is represented as $x_n^{t*}(p_n^t)$.

Theorem 1: The optimal percentage of the CRR's task that is offloaded to price-taking CRP n is

$$x_n^{t*}(p_n^t) = \begin{cases} 0, & \text{if } p_n^t < p_n^{t,\min} \\ \frac{p_n^t}{2\alpha_n^t d}, & \text{if } p_n^{t,\min} \leq p_n^t \leq p_n^{t,\max} \\ 1, & \text{if } p_n^t > p_n^{t,\max} \end{cases}, \quad (9)$$

where $p_n^{t,\min} = 0$ and $p_n^{t,\max} = 2\alpha_n^t d$.

Proof: See Appendix A, available online. $\square$

Theorem 1 shows that the optimal strategy $x_n^{t*}(p_n^t)$ of a price-taking CRP n is a linear and increasing function in p_n^t over interval $[p_n^{t,\min}, p_n^{t,\max}]$. It's important to note that the CRR will not set a price exceeding $p_n^{t,\max}$, as price-taking CRP n can not help process more tasks. Consequently, the CRR's strategy space is constrained to the interval $[p_n^{t,\min}, p_n^{t,\max}]$.

2) *Stage II (CRR):* We proceed to analyze CRR's behavior in Stage II. Given the strategies of price-setting CRPs who set prices (i.e., $\mathbf{p}^s$) and understanding how the CRR's strategy influences the strategies of price-taking CRPs (i.e., $x_n^{t*}(p_n^t), n \in \mathcal{N}^t$), the CRR optimizes its profit during Stage II. This results in the subsequent optimization problem:

$$\max_{\mathbf{x}^s, \mathbf{p}^t, x_0} V((\mathbf{x}^{t*}(\mathbf{p}^t), \mathbf{x}^s), (\mathbf{p}^t, \mathbf{p}^s)) \quad (10)$$

$$\text{s.t. } \sum_{n \in \mathcal{N}^t} x_n^t + \sum_{n \in \mathcal{N}^s} x_n^s + x_0 = 1, \quad (10a)$$

$$0 \leq x_n^s \leq 1, \quad (10b)$$

$$0 \leq x_0 < 1. \quad (10c)$$

If problem (10) is a strictly concave maximization problem, the optimal solution not only exists but is also unique. Given the convexity of all the constraints in problem (10), the concavity of this problem is determined by the concavity of the objective

function (10). In the following theorem, we prove that the objective function in problem (10) is concave.

Theorem 2: Problem (10) is a strictly concave maximization problem, yielding a unique optimal solution under any fixed $\mathbf{p}^s$.

Proof: See Appendix B, available online. $\square$

The optimal solution to problem (10) is denoted as $(\mathbf{x}^{s*}(\mathbf{p}^s), \mathbf{x}^t(\mathbf{p}^s), \mathbf{x}_0^s(\mathbf{p}^s))$. This solution depends on the strategy of price-setting CRPs, i.e., $\mathbf{p}^s$, which will be obtained in Stage I.

3) *Stage I (Price-Setting CRPs):* We then proceed to the analysis of the price-setting CRPs' price competition, each determining their strategies (i.e., the price vector $\mathbf{p}^s$), considering how these strategies would impact the CRR's strategy. The optimal response of the CRR, denoted as $\mathbf{x}^{s*}(\mathbf{p}^s)$, is influenced by both the price p_n^s and the prices submitted by other players. Consequently, we express $\mathbf{p}^s$ as $(p_n^s, \mathbf{p}_{-n}^s)$, where $\mathbf{p}_{-n}^s$ represents the strategy of the price-setting CRPs excluding n . We form the PS-NCG denoted as $\mathcal{G}^s = (\mathcal{N}^s, \mathcal{P}^s, V_n^s)$, where $\mathcal{P}^s$ represents the strategy space.

In order to derive the price-setting CRPs' optimal strategies, we begin by introducing the concepts of best response strategy in the following.

Definition 2: (Best Response Strategy) Given $\mathbf{p}_{-n}^s$, the best response strategy of player n is:

$$p_n^{s*} = \arg \max_{p_n^s \geq 0} V_n^s(x_n^{s*}(p_n^s, \mathbf{p}_{-n}^s), p_n^s) \quad (11)$$

$$\text{s.t. } 0 \leq x_n^{s*}(p_n^s, \mathbf{p}_{-n}^s) \leq 1. \quad (11a)$$

In order to obtain the best response strategies of price-setting CRPs, it is necessary to have knowledge of the CRR's optimal strategy denoted as $x_n^{s*}(p_n^s, \mathbf{p}_{-n}^s)$ in response to the price vector $(p_n^s, \mathbf{p}_{-n}^s)$. We derive the CRR's optimal strategy and its properties using Lemma 1 to subsequently analyze the existence and uniqueness of the NE.

Lemma 1: Given $\mathbf{p}_{-n}^s$, the CRR's optimal strategy is

$$x_n^{s*}(p_n^s, \mathbf{p}_{-n}^s) = \begin{cases} 1, & p_n^s < p_n^{s,\min} \\ 0, & p_n^s > p_n^{s,\max} \\ L_n^s(p_n^s, \mathbf{p}_{-n}^s), & p_n^{s,\min} \leq p_n^s \leq p_n^{s,\max} \end{cases} \quad (12)$$

where $p_n^{s,\min}$ and $p_n^{s,\max}$ are user-dependent constants. $L_n^s(p_n^s, \mathbf{p}_{-n}^s)$ is a function of p_n^s and $\mathbf{p}_{-n}^s$ with

$$L_n^s(p_n^s, \mathbf{p}_{-n}^s) = \frac{1 - A_0 - A^s - A^t}{\sum_{m \in \mathcal{N}^t} \frac{p_n^s}{2\alpha_m^t} + \sum_{m \in \mathcal{N}^s} \frac{p_n^s}{p_m^s} + \frac{p_n^s}{\alpha_0}}, \quad (13)$$

where

$$\begin{aligned} A_0 &= \frac{c}{2\alpha_0 d(N+1)} \left(\frac{1}{f_n^s} - \frac{1}{f_0} \right), \\ A^s &= \sum_{m \in \mathcal{N}^s} \frac{c}{2p_m^s d(N+1)} \left(\frac{1}{f_n^s} - \frac{1}{f_m^s} \right), \\ A^t &= \sum_{m \in \mathcal{N}^t} \frac{c}{4\alpha_m^t d(N+1)} \left(\frac{1}{f_n^s} - \frac{1}{f_m^t} \right). \end{aligned}$$

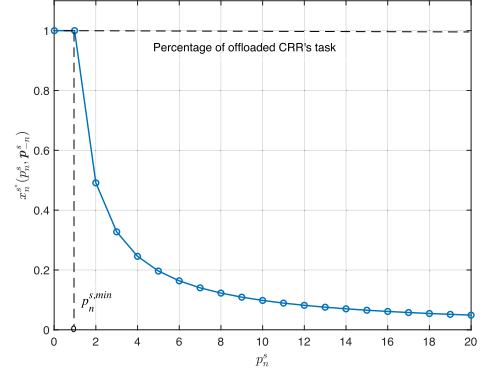


Fig. 3. The CRR's optimal strategy, which determines the optimal percentage of offloaded task $x_n^{s*}(p_n^s, \mathbf{p}_{-n}^s)$ to price-setting CRP $n \in \mathcal{N}^s$. A price-setting CRP n does not submit any price lower than $p_n^{s,\min}$, since the CRR cannot offload more task, and thus setting the price p_n^s below $p_n^{s,\min}$ will decrease its payoff.

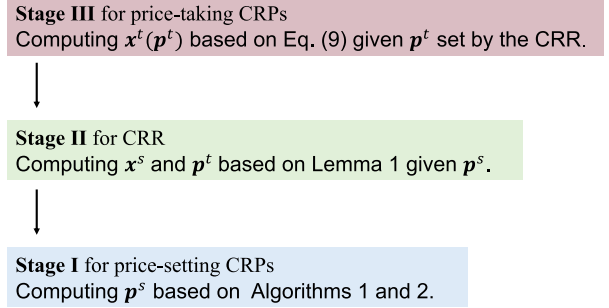


Fig. 4. Logical sequence of the backward induction-based solution to the three-stage Stackelberg game.

Proof: See Appendix C, available online. $\square$

We now derive the properties of the CRR's optimal strategy as expressed in (12).

Lemma 2: Given $\mathbf{p}_{-n}^s$, the CRR's optimal strategy $x_n^{s*}(p_n^s, \mathbf{p}_{-n}^s)$ obtained in (12) is decreasing and convex in p_n^s for $p_n^s \in [p_n^{s,\min}, p_n^{s,\max}]$.

Proof: See Appendix D, available online. $\square$

Fig. 3 illustrates the CRR's optimal percentage of the offloaded task to price-setting CRP n , represented by x_n^{s*} , which confirms that $x_n^{s*}(p_n^s, \mathbf{p}_{-n}^s)$ is decreasing and convex with respect to p_n^s . Based on Fig. 3, it is evident that price-setting CRP n will never set its price p_n^s below $p_n^{s,\min}$, as the CRR offloads the same percentage of the task even when the price is lower than $p_n^{s,\min}$. Thus, setting the price p_n^s below $p_n^{s,\min}$ will not increase the profit of price-setting CRP n .

For clarity, we use Fig. 4 to illustrate the logical sequence of the backward induction-based solution to the three-stage Stackelberg game. In each of the three stages, the CRR and both types of CRPs determine their strategies to maximize their respective profits. We start with Stage III. Given $\mathbf{p}^t$ set by the CRR, price-taking CRPs compute $\mathbf{x}^t$ based on (9). Then for the CRR in Stage II, we determine $\mathbf{x}^s$ utilizing (12). We can also obtain $\mathbf{p}^t$ by following similar steps as outlined in Lemma 1. Finally, price-setting CRPs compute $\mathbf{p}^s$ based on Algorithms 1 and 2, which will be presented in the next section.

B. Equilibrium Analysis

This subsection studies the existence and uniqueness of equilibrium in the PS-NCG.

1) *Existence of Nash Equilibria:* We investigate the PS-NCG formulated in Stage I for price-setting CRPs to determine the existence of a Nash Equilibrium strategy. We first introduce the concept of Nash Equilibrium in the following.

Definition 3: (Nash Equilibrium) A strategy profile $\mathbf{p}^{s^{NE}} = (p_1^{s^{NE}}, \dots, p_{N^s}^{s^{NE}})$ is an NE if it is a fixed point of best responses, i.e., for all $p_n^s \geq 0, n \in \mathcal{N}^s$

$$V_n^s(x_n^{s^*}(p_n^{s^{NE}}, \mathbf{p}_{-n}^{s^{NE}}), p_n^{s^{NE}}) \geq V_n^s(x_n^{s^*}(p_n^{s'}, \mathbf{p}_{-n}^{s^{NE}}), p_n^{s'}).$$

To compute the game's NE, we calculate the best response strategies for the price-setting CRPs within Game PS-NCG by solving problem (11). To this end, we introduce a new variable $\tilde{x}_n^s$ for each price-setting CRP $n \in \mathcal{N}^s$, and replace $x_n^{s^*}(p_n^s, \mathbf{p}_{-n}^s)$ with $\tilde{x}_n^s$. Additionally, we replace p_n^s with $x_n^{s^{*-1}}(\tilde{x}^s)$ in problem (11), where $\tilde{x}^s = (\tilde{x}_n^s)_{n \in \mathcal{N}^s}$. The inverse function $x_n^{s^{*-1}}(\tilde{x}^s)$ always exists, as proven by Lemma 2. Consequently, the strategy space of CRP n is mapped in a one-to-one fashion from $p_n^s \in [p_n^{s, \min}, p_n^{s, \max}]$ into $\tilde{x}_n^s \in [0, 1]$. In this scenario, problem (11) can be transformed into the subsequent equivalent problem, enabling us to derive the best response strategy for CRP n .

$$p_n^{s^*} = \arg \max_{p_n^s \geq 0} V_n^s(\tilde{x}_n^s, x_n^{s^{*-1}}(\tilde{x}^s)) \quad (14)$$

$$\text{s.t. } 0 \leq \tilde{x}_n^s \leq 1. \quad (14a)$$

While the constraint of problem (14) is affine, the objective function may not be concave. We then use Brouwer's fixed point theorem to demonstrate the existence of an NE.

Theorem 3: An NE exists in PS-NCG.

Proof: To prove Theorem 3, we first demonstrate the existence of the fixed point solution of the best response strategies for all price-setting CRPs. In order to find the fixed point solution, we represent CRP n 's best response strategy derived from problem (14) as $\tilde{x}_n^s = f_n(\tilde{x}_{-n}^s)$, with $\tilde{x}_{-n}^s$ representing the offloading percentage of the CRR's task for the price-setting CRPs excluding n . Additionally, we define the function $\tilde{x}^s = \mathbf{F}(\tilde{x}^s)$, where $\mathbf{F} = (f_n)_{n \in \mathcal{N}^s} : [0, 1]_{n \in \mathcal{N}^s} \rightarrow [0, 1]_{n \in \mathcal{N}^s}$.

Brouwer's fixed point theorem [35] asserts that for any continuous function $\mathbf{F}$ mapping a closed convex set into itself, there exists a point $\mathbf{x}$ satisfying $\mathbf{F}(\mathbf{x}) = \mathbf{x}$. Given that the set $[0, 1]$ is both closed and convex, and the function $\mathbf{F} = (f_n)_{n \in \mathcal{N}^s} : [0, 1]_{n \in \mathcal{N}^s} \rightarrow [0, 1]_{n \in \mathcal{N}^s}$ is continuous, a fixed point solution exists for $\mathbf{F}$. This fixed point corresponds to price-setting CRPs' best response strategies obtained from problem (14). In accordance with Definition 3, we establish the existence of an NE. $\square$

2) *Uniqueness of the Nash Equilibrium:* In this subsection, we demonstrate the uniqueness of the NE for Game PS-NCG. In order to derive the NE, we find price-setting CRPs' best response strategies by solving the following problem, by substituting

$x_n^{s^*}(p_n^s, \mathbf{p}_{-n}^s)$ from (12) into problem (11).

$$p_n^{s^*} = \arg \max_{p_n^s \geq 0} (p_n^s - \alpha_n^s) (x_n^{s^*}(p_n^s, \mathbf{p}_{-n}^s))^2 \quad (15)$$

$$\text{s.t. } 0 \leq L_n^s(p_n^s, \mathbf{p}_{-n}^s) \leq 1. \quad (15a)$$

Based on Lemma 2, we observe that constraint (15a) is convex. Subsequently, we proceed to demonstrate the concavity of the objective function. Therefore, problem (15) can be classified as a strictly concave maximization problem, guaranteeing a unique optimal solution.

Lemma 3: There exists $p_n^{s, \max} \geq p_n^{s, \min}$, rendering problem (15) a strictly concave maximization problem within the closed interval $[p_n^{s, \min}, p_n^{s, \max}]$. The objective function decreases when $p_n^s > p_n^{s, \max}$. Consequently, problem (15) possesses a unique optimal solution over the same interval.

Proof: See Appendix E, available online. $\square$

By means of the following theorem, we proceed to establish the uniqueness of the NE in Game PS-NCG.

Theorem 4: A unique NE exists in PS-NCG.

Proof: The proof relies on the subsequent lemma [36]. $\square$

Lemma 4: ([36]) There exists a unique NE in PS-NCG if for all $n \in \mathcal{N}^s$

- The strategy space is a nonempty, convex, and compact subset of some euclidean space.
- Player n 's payoff V_n^s is continuous and strictly concave in its strategy p_n^s .

As stated in Lemma 4, PS-NCG fulfills the described properties, allowing us to ascertain the NE's uniqueness.

V. ALGORITHM DESIGN

In this section, we design iterative algorithms for the CRR and the price-setting CRPs in the profit maximization problem. Then we provide a demonstration and proof of their convergence to the unique NE.

The iterative algorithms for both the CRR and price-setting CRPs are designed to converge to the PS-NCG's NE. These algorithms allow price-setting CRPs to update their strategies without needing complete information about the CRR or other CRPs. Moreover, the designed algorithm for the CRR supports decentralized information exchange among the price-setting CRPs.

Initially, we devise an iterative algorithm for the price-setting CRPs to update their best response strategies. Let $x_n^{s^{(t)}}$ and $p_n^{s^{(t)}}$ represent the CRR's and CRP n 's strategies at the t -th iteration. Considering a price-setting CRP $n \in \mathcal{N}^s$, whose best response strategy can be determined by solving problem (15), the CRP n 's strategy can be updated using the following rule based on the gradient method.

$$p_n^{s^{(t+1)}} = \text{Pro} \left(p_n^{s^{(t)}} + \eta^{(t)} x_n^{s^{(t)}} \times \left[x_n^{s^{(t)}} + 2 \left(p_n^{s^{(t)}} - \alpha_n^s \right) \frac{\partial x_n^{s^*}}{\partial p_n^s} \Big|_{p_n^s = p_n^{s^{(t)}}} \right] \right), \quad (16)$$

where $\text{Pro}(\cdot)$ represents the projection onto the feasible region of problem (15), and $\eta^{(t)}$ denotes the step size at iteration t .

Algorithm 1: Iterative Best Response Adaptation for a Price-Setting CRP n .

```

1: initialization:  $t \leftarrow 0$ ,  $p_n^{s(0)}$ , and  $\epsilon$ .
2: while  $|p_n^{s(t)} - p_n^{s(t-1)}| > \epsilon$  do
3:   CRP  $n$  submits the price strategy  $p_n^{s(t)}$  to the CRR.
4:   CRP  $n$  collects the CRR's responses, i.e.,
        $x_n^{s*}(p_n^s, p_{-n}^s)$  and  $\frac{\partial x_n^{s*}}{\partial p_n^s}$ .
5:   The CRP updates its price strategy based on (16).
6:    $t \leftarrow t + 1$ .
7: end while
8: output:  $p_n^{s(t)}$ 

```

Algorithm 2: Distributed Iterative Algorithm for Information Exchange Among the CRR and CRPs.

```

1: initialization:  $k \leftarrow 0$ , conver_flag  $\leftarrow 0$ , and  $\epsilon'$ .
2: while conver_flag = 0 do
3:   The CRR collects  $p_n^{s(k)}$  from all CRPs  $n \in \mathcal{N}^s$ .
4:   Each price-setting CRP updates its best response
       strategy via Algorithm 1.
5:   The CRR checks the termination criterion.
6:   if  $|p_n^{s(k+1)} - p_n^{s(k)}| < \epsilon', \forall n \in \mathcal{N}^s$  then
7:     conver_flag  $\leftarrow 1$ .
8:   end if
9:    $k \leftarrow k + 1$ .
10: end while
11: output:  $p^{s(k)}$ 

```

We present the gradient-based algorithm for price update at price-setting CRPs in Algorithm 1. Initially, price-setting CRP n randomly initializes $p_n^{s(0)}$ (Step 1). CRPs then submit their prices to the CRR (Step 3). Simultaneously, the CRR computes $x_n^{s*}(p_n^s, p_{-n}^s)$ and $\partial x_n^{s*} / \partial p_n^s$ and communicates them to the CRR. Upon receiving the CRR's responses (Step 4), the CRP updates the price using rule (16) (Step 5). This process continues until convergence. The CRP verifies the termination criterion (Step 2) and terminates the algorithm when the relative changes of the prices between two consecutive iterations are sufficiently small (as defined by the positive constant ϵ). By employing this iterative algorithm, price-setting CRPs only require knowledge of the CRR's responses to their strategies. The proposed iterative algorithm incurs a relatively small communication overhead, as only a few messages need to be exchanged.

Employing Algorithm 2, the CRR facilitates information exchange among the price-setting CRPs. The CRR's response to each price-setting CRP reflects the strategies of other CRPs. Consequently, the price-setting CRPs do not need to know the strategies employed by their counterparts. The CRR shares the necessary information with the price-setting CRPs and obtains their updated best response strategies during each iteration. This process continues until convergence.

We now establish the convergence of our algorithms. As problem (15) is a strictly concave maximization problem, which has been shown in Lemma 3, the gradient-based method in Algorithm 1 converges to the optimal solution of this problem.

Next, we adopt the same rationale as presented in the proof of [37] to investigate the convergence of Algorithm 2.

Theorem 5: The iterative algorithm proposed in Algorithm 2 converges to the unique NE of Game PS-NCG.

Proof: See Appendix F, available online. $\square$

Note that the gradient descent-based method in Algorithm 1 converges to the optimal solution of this problem using $O(\log(1/\epsilon))$ iterations [38], where ϵ represents the termination threshold in Algorithm 1. The primary arithmetic operations in Algorithm 1 is computing $\partial x_n^{s*} / \partial p_n^s$. Hence, the computational complexity for one iteration in Algorithm 1 is $O((N_s))$. Consequently, the total computational complexity for Algorithm 1 is $O((N_s)\log(1/\epsilon))$. In Algorithm 2, each price-setting CRP updates its best response strategy using Algorithm 1. Therefore, the total computational complexity for Algorithm 2 is $O((N_s)^2 \log(1/\epsilon) T_2)$, where T_2 represents the number of iterations required by Algorithm 2 to achieve the accuracy ϵ' .

VI. PERFORMANCE EVALUATION

In this section, we conduct comprehensive numerical studies to evaluate the CRR's profit when offloading its task to the CRPs. As indicated in Section I, none of the existing works considered the same problem setting as this paper. Hence, there is no suitable existing benchmark to compare with. Instead, we compare our derived SPE with the outcome of a splittable task offloading marketplace where the CRR only incentivizes price-setting CRPs to assist in processing the CRR's task [8].

Furthermore, we perform a comparison between our proposed framework, where the CRR offloads its task to both price-setting and price-taking CRPs, with the socially optimal solution to assess the SPE's efficiency. The SPE's efficiency is defined as the ratio between the SPE's social welfare and the maximum social welfare. This value, constrained to be no more than 1, presents how the market outcome deteriorates due to the players' selfishness considered in this paper. We define the sum of all players' payouts as social welfare.

$$\Phi(\mathbf{x}) = cT_s - \alpha_0(x_0d)^2 - \sum_{n \in \mathcal{N}^t} \alpha_n^t(x_n^t d)^2 - \sum_{n \in \mathcal{N}^s} \alpha_n^s(x_n^s d)^2. \quad (17)$$

The maximum social welfare Φ^* can be obtained by solving the following problem.

$$\Phi^* = \max_{\mathbf{x} \in \mathcal{R}} \Phi(\mathbf{x}), \quad (18)$$

where $\mathcal{R}$ implies $\mathbf{x}$'s feasible region. Denote the equilibrium strategies derived in our framework as $\mathbf{x}^{NE}$, we present the following definition of SPE's efficiency.

Definition 4: (SPE's Efficiency) The SPE's efficiency is defined as the ratio between the SPE's social welfare and the maximum social welfare, i.e.,

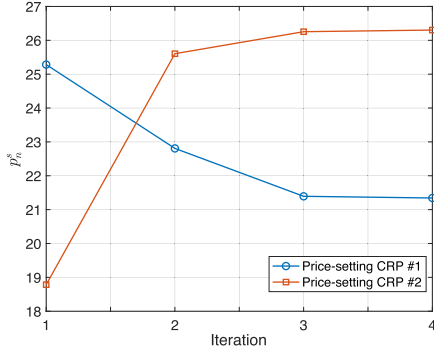
$$\frac{\Phi(\mathbf{x}^{NE})}{\Phi^*}.$$

A. Experimental Setup

We consider a scenario where a self-driving car (e.g., the CRR) processes real-time video for object detection. Due to the

TABLE II
PARAMETER SETTINGS

Simulation parameters	Values
Number of CRPs	[3 – 10]
Number of price-setting CRPs	[2 – 8]
CPU frequency of price-setting CRPs	5 GHz
Cost factor of price-setting CRPs α_n^s	[8 – 16]
CPU frequency of price-taking CRPs	2 GHz
Cost factor of price-taking CRPs α_n^t	[3 – 4]
CPU frequency of the CRR	1 GHz
Cost factor of the CRR α_0	[1 – 2]
Workload d	[0.5 – 4] GHz
Revenue constant c	[6 – 12]

Fig. 5. The prices chosen by price-setting CRPs in different iterations as obtained by Algorithm 2 ($\epsilon = 10^{-6}$, $\epsilon' = 0.01$, and $\eta^{(t)} = 0.1$).

limited computing capacity, the car can offload tasks to external CRPs for collaborative execution. The price-setting CRPs can be roadside unit (RSU) edge servers owned by edge service providers such as China Mobile and Unicom. Peer vehicles with excess computing capacity can act as price-taking CRPs. Unless stated otherwise, there are a total of 5 CRPs, comprising 3 price-taking CRPs and 2 price-setting CRPs. It's important to note that we set the computing power and cost factor of CRPs and the CRR based on prior studies [39], [40], [41]. The cost factors of CRR, price-setting CRPs, and price-taking CRPs are randomly sampled from the uniform distribution $\mathbb{U}(1, 2)$, $\mathbb{U}(3, 4)$, and $\mathbb{U}(8, 16)$. The termination thresholds ϵ and ϵ' in Algorithms 1 and 2 are set to 10^{-6} and 0.01, and the step size $\eta^{(t)}$ for price update in (16) is fixed at 0.1. We summarize all the simulation parameters in Table II unless specified otherwise.

B. Experimental Results

1) *Convergence Behavior*: We investigate the convergence of Algorithm 2 in Fig. 5, considering 5 CRPs, including 2 price-setting CRPs. During each iteration, the price-setting CRPs update their best response strategies and submit them to the CRR. Fig. 5 presents the price dynamics chosen by the price-setting CRPs, illustrating the rapid convergence of Algorithm 2. We conducted further convergence experiments with varying numbers of price-setting CRPs, specifically when the number of CRPs is set to 5, as shown in the Table III. The results indicate that the number of iterations required for convergence increases slightly as the number of price-setting CRPs increases.

TABLE III
CONVERGENCE PERFORMANCE WITH VARYING NUMBER OF PRICE-SETTING CRPs

Number of price-setting CRPs	2	3	4
Number of iteration round	2.98	3.68	3.87

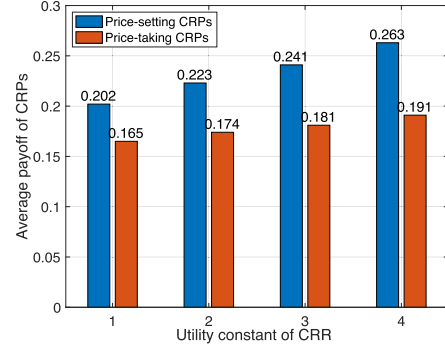


Fig. 6. Average profits for price-setting and price-taking CRPs versus the CRR's revenue constant.

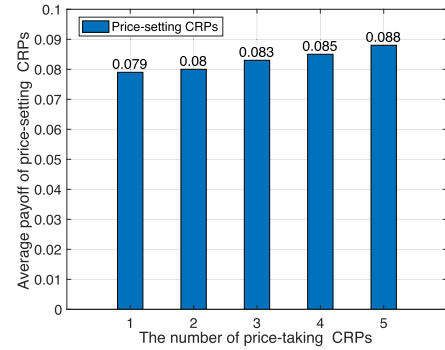


Fig. 7. Average profit that a price-setting CRP can obtain for different number of price-taking CRPs.

Given Theorem 5's assertion of the convergence of this algorithm to the unique NE of PS-NCG, we conclude that Algorithm 2 rapidly converges to the SPE.

We now examine the payoffs attained by the CRPs when assisting in the processing of offloaded tasks. In order to assess the payoffs of various CRPs equitably, we adopt an identical set of parameters for all CRPs. Fig. 6 depicts the average payoffs per CRP acquired by both price-setting and price-taking CRPs. It is evident that a higher revenue constant of the CRR results in increased payoff potential for the CRPs. Moreover, price-setting CRPs demonstrate the ability to achieve higher average payoffs compared to price-taking CRPs, attributed to their greater market power.

We proceed to investigate how the number of price-taking CRPs influences the payoffs of price-setting CRPs. Fig. 7 illustrates the average payoffs of price-setting CRPs as the number of price-taking CRPs varies in the edge computing market, with a total of 10 CRPs. As depicted in the figure, the involvement of price-taking CRPs results in fewer price-setting CRPs, thereby diminishing price competition among them. Consequently, price-setting CRPs achieve higher payoffs.

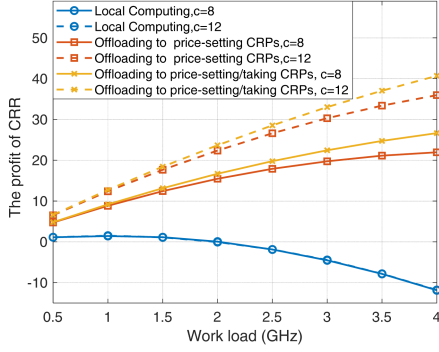


Fig. 8. The profit of CRR under varying workloads.

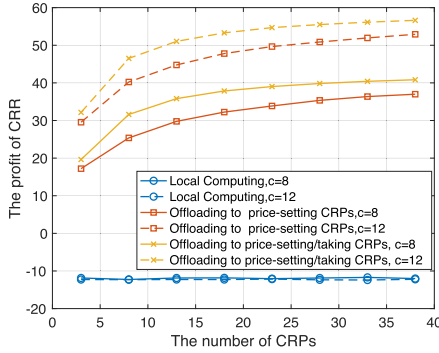


Fig. 9. The profit of CRR under a varying number of CRPs.

2) *CRR's Profit*: We present the profit of CRR under varying workloads in Fig. 8. It is evident that offloading the CRR's task to both price-taking and price-setting CRPs yields higher profit compared to offloading the task to only price-setting CRPs and performing local computing. In cases where the workload is excessively high, task execution on the local device results in minimal profit. This is attributed to the CRR's limited computing power; processing a computation-intensive task incurs a substantial cost, and the revenue obtained from this task cannot cover the CRR's cost. The revenue factor c is unrelated to local computing. Hence, altering the revenue factor does not affect the CRR's profit with local computing. As the revenue factor c increases, the CRR's profit achieved by offloading to CRPs also increases.

We illustrate the profit of CRR under a varying number of CRPs in Fig. 9 while keeping the number of price-setting CRPs fixed at 2. Similarly, offloading the task to both price-taking and price-setting CRPs yields greater profit for the CRR than offloading the task to only price-setting CRPs or performing local computing. As the number of CRPs is not related to local computing, changing the number of CRPs will have no effect on the CRR's profit for local computing. When the number of CRPs increases, the CRR's profit achieved by offloading to CRPs also increases. Moreover, the increasing number of CRPs compromises the performance gain of offloading the task to both price-taking and price-setting CRPs.

We depict the profit of CRR under a varying number of price-setting CRPs in Fig. 10 while keeping the total number of CRPs fixed at 30. For the benchmark where the CRR offloads

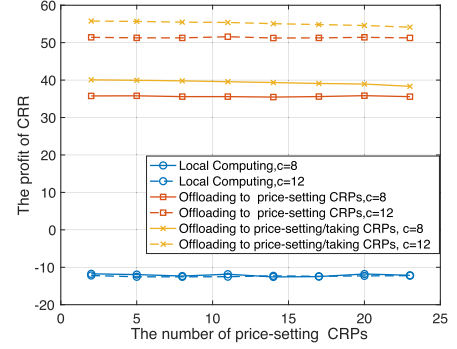


Fig. 10. The profit of CRR under a varying number of price-setting CRPs, with the total number of CRPs being fixed at 30.

the task to only price-setting CRPs, the number of CRPs, i.e., the price-setting CRPs in this case, is also fixed at 30, and the CRR's profit remains constant. Note that in our proposed framework, as the number of price-setting CRPs increases, the number of price-taking CRPs decreases, resulting in a reduced gain introduced by using price-taking CRPs. Consequently, the CRR's profit from offloading the CRR's task to both price-taking and price-setting CRPs decreases as the number of price-setting CRPs increases with a fixed total number of CRPs. Additionally, the number of price-setting CRPs is unrelated to local computing, so changing this number will not affect the CRR's profit with local computing.

3) *The Efficiency of the SPE*: In this part, we evaluate the SPE's efficiency. Fig. 11 presents the ratio between the social welfare obtained at an SPE and the maximum social welfare, which is obtained through optimization via CVX [42]. It is evident that our proposed framework's social welfare approaches the maximum social welfare under varying numbers of CRPs, price-setting CRPs, and workload. Moreover, an increase in the revenue factor c results in a slightly higher efficiency of the SPE. Notably, when the number of CRPs increases, the efficiency of the SPE also increases. Conversely, an increase in the number of price-setting CRPs and workload leads to a decrease in the efficiency of the SPE. One primary reason for this phenomenon is that a higher proportion of the task being processed locally results in less competition and, thus, a lower efficiency of the SPE. Next, we will illustrate the proportion of the task processed locally or at CRPs in Fig. 12 to corroborate this.

We illustrate the proportion of the task processed locally or at CRPs in Fig. 12. We have three-fold observations:

- It is evident from Fig. 12(a) that as the workload increases, the proportion of the task processed by CRPs decreases. This is likely due to the cost being a quadratic function of the workload. As the cost factor is proportional to the computing power of devices, an increase in workload leads to higher costs for CRPs. Consequently, a larger workload results in a smaller portion of the task being processed by CRPs. Correspondingly, a smaller portion of the task being processed by CRPs degrades the efficiency of SPE, as depicted in Fig. 11(a).
- Fig. 12(b) presents the proportion of the task processed locally or at CRPs under varying numbers of CRPs. With

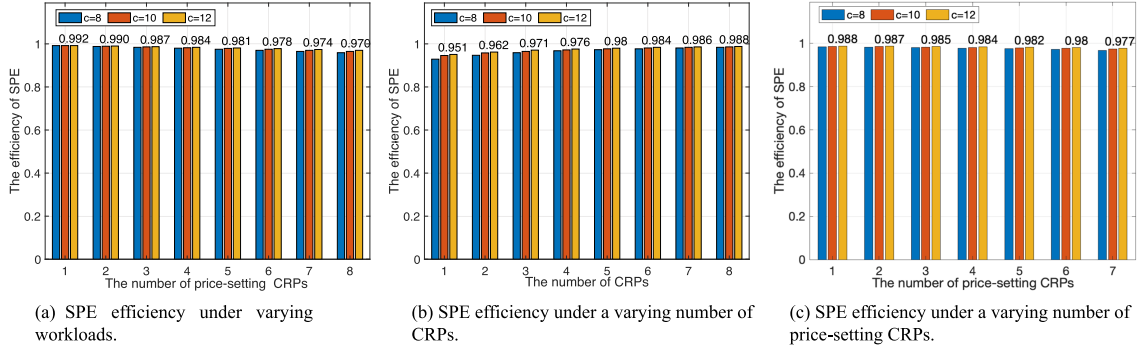


Fig. 11. SPE efficiency comparison. The proposed three-stage framework achieves a close-to-optimal social welfare. The data in figures denote the performance when $c = 12$.

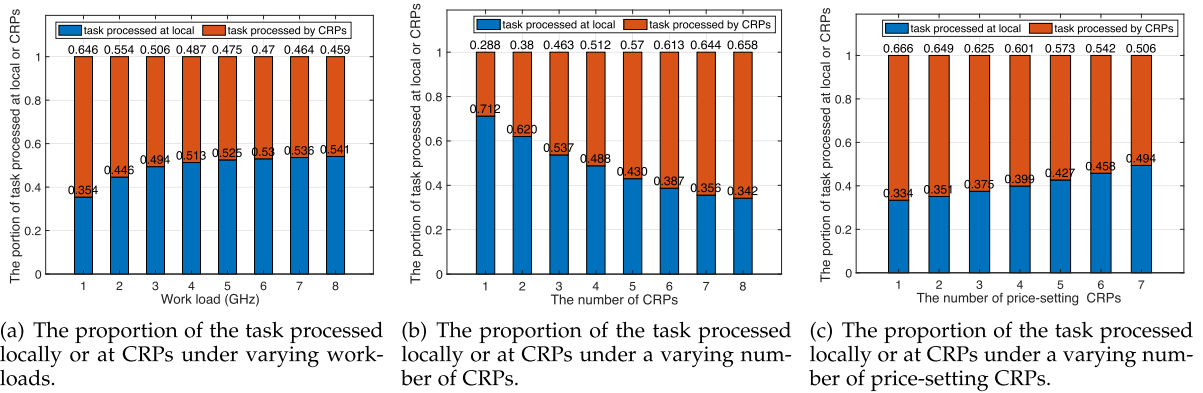


Fig. 12. The proportion of the task processed locally or at CRPs.

the number of price-setting CRPs fixed at 2, an increasing number of CRPs implies a greater number of price-taking CRPs. This results in an increase in the CRR's profit due to the gain introduced by adopting price-taking CRPs in comparison to using only price-setting CRPs. Therefore, the proportion of the task processed by CRPs increases as the number of CRPs increases. This leads to a higher efficiency of SPE with a larger number of CRPs when the number of price-setting CRPs is fixed, which corresponds to Fig. 11(b).

- Fig. 12(c) shows the proportion of the task processed locally or at CRPs under varying numbers of price-setting CRPs. With the total number of CRPs fixed at 10, an increasing number of price-setting CRPs implies a decrease in the number of price-taking CRPs. Consequently, the CRR's profit achieved by using price-taking CRPs diminishes, leading to a larger portion of the task being processed by local computing. Accordingly, a larger portion of the task being processed by local computing degrades the efficiency of SPE, as shown in Fig. 11(c).

VII. CONCLUSION

This paper presented an incentive framework for splittable task offloading that includes both price-setting and price-taking CRPs, with the goal of maximizing the profit for both the CRR

and the CRPs. The interactions between the CRR and these CRPs are modeled as a three-stage Stackelberg game. We prove the existence of a unique subgame perfect equilibrium and develop iterative algorithms for the CRR and price-setting CRPs to reach this equilibrium. Extensive numerical simulations corroborate that offloading tasks to both price-taking and price-setting CRPs can yield higher profits for the CRR and price-setting CRPs compared to offloading only to price-setting CRPs. Additionally, the social welfare at the subgame perfect equilibrium in our proposed framework closely approaches the socially optimal solution.

In our future work, we plan to explore task offloading marketplaces where multiple CRRs offload their splittable tasks to both price-taking and price-setting CRPs. The models of the CRR and CRPs presented in this paper remain applicable, with additional considerations on the competition among CRRs for communication and computation resources during task offloading. This substantially complicates the characterization of interactions between CRRs and CRPs.

REFERENCES

- [1] Z. Wang, K. Liu, J. Hu, J. Ren, H. Guo, and W. Yuan, "Attrleaks on the edge: Exploiting information leakage from privacy-preserving co-inference," *Chin. J. Electron.*, vol. 32, no. 1, pp. 1–12, 2023.
- [2] Y. Yang et al., "6G network AI architecture for everyone-centric customized services," *IEEE Netw.*, vol. 37, no. 5, pp. 71–80, Sep. 2023.

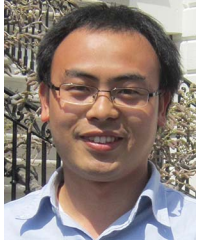
- [3] H. Wu, H. Tian, S. Fan, and J. Ren, "Data age aware scheduling for wireless powered mobile-edge computing in Industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 17, no. 1, pp. 398–408, Jan. 2021.
- [4] Z. Ni, H. Chen, B. Gao, K. Lin, L. Wu, and J. Yu, "Reward-oriented task offloading in energy harvesting collaborative edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14414–14426, Dec. 2024.
- [5] M. Ma, C. Gong, L. Wu, and Y. Yang, "FLIRAS: Fast learning with integrated reward and reduced action space for online multitask offloading," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 5406–5417, Mar. 2023.
- [6] S. Xia, Z. Yao, Y. Li, and S. Mao, "Online distributed offloading and computing resource management with energy harvesting for heterogeneous MEC-Enabled IoT," *IEEE Trans. Wireless Commun.*, vol. 20, no. 10, pp. 6743–6757, Oct. 2021.
- [7] L. Wu, G. Gao, J. Yu, F. Zhou, Y. Yang, and T. Wang, "PDD: Partitioning DAG-topology DNNs for streaming tasks," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 9258–9268, Mar. 2024.
- [8] Z. Chen, Q. Ma, L. Gao, and X. Chen, "Price competition in multi-server edge computing networks under SAA and SIQ models," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 754–768, Jan. 2024.
- [9] H. Zhou, T. Wu, H. Zhang, and J. Wu, "Incentive-driven deep reinforcement learning for content caching and D2D offloading," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2445–2460, Aug. 2021.
- [10] X. Pu et al., "Incentive mechanism and resource allocation for collaborative task offloading in energy-efficient mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 72, no. 10, pp. 13775–13780, Oct. 2023.
- [11] G. Li and J. Cai, "An online incentive mechanism for collaborative task offloading in mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 624–636, Jan. 2020.
- [12] M. Liu and Y. Liu, "Price-based distributed offloading for mobile-edge computing with computation capacity constraints," *IEEE Wireless Commun. Lett.*, vol. 7, no. 3, pp. 420–423, Jun. 2018.
- [13] M. Tao, K. Ota, M. Dong, and H. Yuan, "Stackelberg game-based pricing and offloading in mobile edge computing," *IEEE Wireless Commun. Lett.*, vol. 11, no. 5, pp. 883–887, May 2022.
- [14] Z. Tong, X. Deng, J. Mei, L. Dai, K. Li, and K. Li, "Stackelberg game-based task offloading and pricing with computing capacity constraint in mobile edge computing," *J. Syst. Archit.*, vol. 137, 2023, Art. no. 102847.
- [15] Q. Wang, S. Guo, J. Liu, C. Pan, and L. Yang, "Profit maximization incentive mechanism for resource providers in mobile edge computing," *IEEE Trans. Services Comput.*, vol. 15, no. 1, pp. 138–149, Jan./Feb. 2022.
- [16] T. Wang, Y. Lu, J. Wang, H. -N. Dai, X. Zheng, and W. Jia, "EIHDP: Edge-intelligent hierarchical dynamic pricing based on cloud-edge-client collaboration for IoT systems," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1285–1298, Aug. 2021.
- [17] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient multi-user computation offloading for mobile-edge cloud computing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, Oct. 2016.
- [18] Z. Kuang, L. Li, J. Gao, L. Zhao, and A. Liu, "Partial offloading scheduling and power allocation for mobile edge computing systems," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6774–6785, Aug. 2019.
- [19] Y. Wang et al., "A game-based computation offloading method in vehicular multiaccess edge computing networks," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 4987–4996, Jun. 2020.
- [20] G. Zhang, F. Shen, Z. Liu, Y. Yang, K. Wang, and M. Zhou, "FEMTO: Fair and energy-minimized task offloading for fog-enabled IoT networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4388–4400, Jun. 2019.
- [21] H. Ko and S. Pack, "Distributed device-to-device offloading system: Design and performance optimization," *IEEE Trans. Mobile Comput.*, vol. 20, no. 10, pp. 2949–2960, Oct. 2021.
- [22] T. Zhang and W. Chen, "Computation offloading in heterogeneous mobile edge computing with energy harvesting," *IEEE Trans. Green Commun. Netw.*, vol. 5, no. 1, pp. 552–565, Mar. 2021.
- [23] Z. Liu, Y. Yang, K. Wang, Z. Shao, and J. Zhang, "POST: Parallel offloading of splittable tasks in heterogeneous fog networks," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3170–3183, Apr. 2020.
- [24] Y. Peng, X. Song, F. Liu, G. Xing, and T. Song, "Joint task partition and computation offloading for latency-sensitive services in mobile edge networks," in *Proc. 18th Int. Conf. Mobility, Sens. Netw.*, Guangzhou, China, 2022, pp. 191–196.
- [25] T. Q. Dinh, Q. D. La, T. Q. S. Quek, and H. Shin, "Learning for computation offloading in mobile edge computing," *IEEE Trans. Commun.*, vol. 66, no. 12, pp. 6353–6367, Dec. 2018.
- [26] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4377–4387, Jun. 2019.
- [27] Q. Wang, S. Guo, J. Liu, C. Pan, and L. Yang, "MotiShare: Incentive mechanisms for content providers in heterogeneous time-varying edge content market," *IEEE Trans. Services Comput.*, vol. 16, no. 1, pp. 452–465, Jan./Feb. 2023.
- [28] Q. Wang, S. Guo, G. Liu, L. Yang, and C. Pan, "MotiLearn: Contract-based incentive mechanism for heterogeneous edge collaborative training," *IEEE Trans. New. Sci. Eng.*, vol. 9, no. 4, pp. 2895–2909, Jul./Aug. 2022.
- [29] H. S. Mansouri, V. W. S. Wong, and J. Huang, "An incentive framework for mobile data offloading market under price competition," *IEEE Trans. Mobile Comput.*, vol. 16, no. 11, pp. 2983–2999, Nov. 2017.
- [30] P. Sun, L. Wu, Z. Wang, J. Liu, J. Luo, and W. Jin, "A profit-maximizing data marketplace with differentially private federated learning under price competition," in *Proc. ACM Manag. Data*, vol. 2, Sep. 2024, Art. no. 191.
- [31] Y. Zhang and Y. Chen, "Optimal nonlinear pricing in social networks under asymmetric network information," *Operations Res.*, vol. 68, no. 3, pp. 818–833, 2020.
- [32] Y. Li, B. Yang, H. Wu, Q. Han, C. Chen, and X. Guan, "Joint offloading decision and resource allocation for vehicular fog-edge computing networks: A contract-stackelberg approach," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15969–15982, Sep. 2022.
- [33] M. Wang, L. Zhang, P. Gao, X. Yang, K. Wang, and K. Yang, "Stackelberg-game-Based intelligent offloading incentive mechanism for a Multi-UAV-Assisted mobile-edge computing system," *IEEE Internet Things J.*, vol. 10, no. 17, pp. 15679–15689, Sep. 2023.
- [34] A. Mas-Colell, M. D. Whinston, and J. R. Green, *Microeconomic Theory*. New York, NY, USA: Oxford Univ. Press, 1995.
- [35] Y. Shoham and K. Brown, *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge, U.K.: Cambridge Univ. Press, 2008.
- [36] J. B. Rosen, "Existence and uniqueness of equilibrium points for concave n-person games," *Econometrica*, vol. 33, no. 3, pp. 347–351, Jun. 1965.
- [37] R. D. Yates, "A framework for uplink power control in cellular radio systems," *IEEE J. Sel. Areas Commun.*, vol. 13, no. 7, pp. 1341–1347, Sep. 1995.
- [38] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [39] L. Wu, P. Sun, Z. Wang, Y. Li, and Y. Yang, "Computation offloading in multi-cell networks with collaborative edge-cloud computing: A game theoretic approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 3, pp. 2093–2106, Mar. 2024.
- [40] Y. Yang, Z. Liu, X. Yang, K. Wang, X. Hong, and X. Ge, "POMT: Paired offloading of multiple tasks in heterogeneous fog networks," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8658–8669, Oct. 2019.
- [41] K. Wang, Y. Zhou, J. Li, L. Shi, W. Chen, and L. Hanzo, "Energy-efficient task offloading in massive MIMO-Aided multi-pair fog-computing networks," *IEEE Trans. Commun.*, vol. 69, no. 4, pp. 2123–2137, Apr. 2021.
- [42] M. Grant and S. Boyd, "CVX: Matlab software for disciplined convex programming, version 2.1," 2014. [Online]. Available: <http://cvxr.com/cvx>



Liantao Wu received the BE degree in automation from Shandong University, China, in 2012 and the PhD degree in control science and engineering from Zhejiang University, China, in 2017. He is currently an associate professor with East China Normal University, China. His research interests include IoT, edge computing, and edge intelligence.



Peng Sun received the BE degree in automation from Tianjin University, China, in 2015, and the PhD degree in control science and engineering from Zhejiang University, China, in 2020. He is currently an associate professor with the College of Computer Science and Electronic Engineering, Hunan University, China. His research interests include the Internet of Things, mobile crowdsensing, and federated learning.



Zhibo Wang (Senior Member, IEEE) received the BE degree in automation from Zhejiang University, China, in 2007, and the PhD degree in electrical engineering and computer science from the University of Tennessee, Knoxville, in 2014. He is currently a professor with the College of Computer Science and Technology, Zhejiang University. His current research interests include mobile crowdsensing systems, cyber-physical systems, recommender systems, and privacy protection. He is a member of ACM.



Honglong Chen (Senior Member, IEEE) received the ME degree in control science and engineering from Zhejiang University, China, in 2008, and the PhD degree in computer science from The Hong Kong Polytechnic University, Hong Kong, in 2012. He was a postdoctoral researcher with the School of CIDSE, Arizona State University from 2015 to 2016. He is currently a professor and PhD supervisor with the College of Control Science and Engineering, China University of Petroleum, China. He served as a guest editor of *IEEE Transactions on Industrial Informatics*. His current research interests are in the areas of Internet of Things and cyber security. He is a senior member of CCF (China Computer Federation) and a member of ACM.



Xiaoyi Pang received the BE degree in information security from Wuhan University, China, in 2018. She is currently working toward the PhD degree with the School of Cyber Science and Engineering, Wuhan University. Her research interest focuses on privacy protection in mobile crowdsensing systems and edge intelligence.



Yang Yang (Fellow, IEEE) received the BS and MS degrees in radio engineering from Southeast University, Nanjing, China, in 1996 and 1999, respectively, and the PhD degree in information engineering from the Chinese University of Hong Kong, in 2002. He is currently the associate vice-president for teaching and learning, the acting dean with the College of Education Sciences, and a professor with the IoT Thrust, Hong Kong University of Science and Technology (Guangzhou), China. His research interests include fog computing networks, service-oriented collaborative intelligence, wireless sensor networks, IoT applications, and advanced testbeds and experiments. He has published more than 280 papers and filed more than 80 technical patents in these research areas. He has been the chair of the Steering Committee of Asia-Pacific Conference on Communications (APCC) since January 2019. In addition, he is a general co-chair of the IEEE DSP 2018 conference and a TPC vice-chair of the IEEE ICC 2019 conference.



Jiahui Hu received the MS degree in cyber security from Wuhan University, China, in 2019. She is currently working toward the doctor degree with the School of Cyber Science and Engineering, Zhejiang University. Her research interest focuses on federated learning and privacy.